# Contents- Unit-III
# Introduction to Cryptography & Cryptocurrencies

**Introduction to Cryptography & Cryptocurrencies**:

- Cryptographic Hash Functions
- SHA-256
- Hash Pointers and Data Structures
- Digital Signatures
- Public Keys as Identities
- A Simple Cryptocurrency

**DLT -- Decentralized database maintained across multiple nodes, ensuring security, transparency, and trust.**

**It underpins technologies like blockchain and cryptocurrencies. Key components include**:

- Cryptographic Hash Functions
- SHA-256
- Hash Pointers and Data Structures
- Digital Signatures
- Public Keys as Identities

Let’s see all key components in brief one by one

## 1. Cryptographic Hash Functions

- H**ash function** is a one-way algorithm that takes an input and produces a fixed-size output
- Even a small change in input results in a completely different hash, ensuring **data integrity**.
- Used for verifying transactions and linking blocks in a **blockchain**.
- Example: **SHA-256** (Secure Hash Algorithm 256-bit).

## 2. SHA-256

- Widely used cryptographic hash function, especially in **Bitcoin and blockchain**.
- Generates a **256-bit (32-byte) hash** from any input.
- Provides **collision resistance** (two different inputs don’t produce the same hash) and **pre-image resistance** (hashes can’t be reversed to find the original input).
- Used in **proof-of-work (PoW)** mining to secure transactions

## 3. Hash Pointers and Data Structures

- H**ash pointer** is a pointer to the location of data, but instead of just pointing, it also stores the **hash of that data**.
- This ensures **tamper-proof linking** of data blocks, forming the basis of blockchain's **linked list structure**.

**Example: A blockchain consists of blocks, where each block contains:**

- **Data (transactions)**
- **Hash pointer** to the previous block
- **Timestamp and nonce**

Any modification in a previous block would change its hash, making it evident that data was altered.

**Hash is just a fixed-size output, while a hash pointer combines a pointer with a hash to link and secure data.**

## 4. Digital Signatures

- Cryptographic technique that provides **authentication, data integrity, and non-repudiation**.
- Uses **asymmetric cryptography** (public and private key pairs).

**Process:**

Sender signs the transaction using their **private key**.

Receiver verifies the signature using the sender's **public key**.

Ensures that transactions cannot be **forged or tampered** with.

Used in cryptocurrency transactions to prove ownership and legitimacy.

## 5. Public Keys as Identities

In decentralized systems, users are identified by their **public key**, rather than personal details.

A user has a **public-private key pair**:

**Public key** (shared openly): Acts as an identity.

**Private key** (kept secret): Used to sign transactions.

**Example in cryptocurrencies**:

A Bitcoin wallet address is derived from the **public key**.

Only the person with the **private key** can spend the associated funds.

# Old Traditional Financial Arrangements

Let’s say Alice wants a tool and Bob wants medicine

let’s say Alice has food that she’s willing to trade for a tool, while Bob, who has a tool, doesn’t have any need for food. He wants medicine instead. Alice and Bob can’t trade with each other, but if there’s a third person, Carol, who has medicine that she’s willing to trade for food, then it becomes possible to arrange a three-way swap where everyone gets what they need

**Drawback, of course, is coordination**

Arranging a group of people, whose needs and wants align, in the same place at the same time

**So What is the solution ?**

**Credit and Cash**

# Introduction to Cryptography & Cryptocurrencies

All currencies (Fiat or digital) : Control and security essential to prevent cheating.

In fiat currencies, organizations like central banks control the money supply and add anti-counterfeiting features to physical currency.

These security features raise bar for attacker, but don't make money impossible to counterfeit.

Ultimately, law enforcement is necessary for stopping people from breaking rules of system

But unlike fiat currencies, security rules of cryptocurrencies need to be enforced purely **technologically** and without relying on a central authority.

**Cryptocurrencies make heavy use of cryptography which helps to secure this currency.**

**In this unit we will explore two important primitives cryptographic hashes and digital signatures those help to build cryptocurrencies**

# Cryptographic Hash Functions

**First cryptographic primitive ---Cryptographic Hash Function**

**Cryptographic Hash Function** -- characterized by three key attributes:

**Flexibility in input**: They can accept strings of any length.

**Consistency in output**: They consistently yield outputs of a fixed size.

**Efficiency in computation**: Can be efficiently calculated, implying that determining output within a reasonable timeframe.

However, to ensure cryptographic robustness, hash functions must possess three additional **properties:(1) collision-resistance, (2) hiding, (3) puzzle-friendliness**

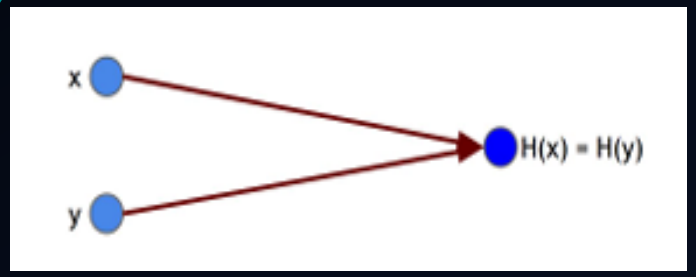
# Cryptographic Hash Functions: Collision-Resistance Property

## Collision-Resistance:

- Hash function prevents finding two inputs with the same output.
- Collisions exist but are hard to find.
- No hash is proven collision-resistant; we trust those unbroken so far.

## Application: Message Digests:

- Hashes verify data integrity (e.g., file upload/download).
- Compare hash of original and downloaded file to detect tampering.
- Fixed-length hash saves storage and ensures security.




Hash collision: x and y are distinct values, yet when input into hash function produce same output.

# Cryptographic Hash Functions

Hash function is a integral  and very important part of blockchain data structure ,

**It connects various blocks in blockchain.**

**One-way function : Converts input data of arbitrary length and produces a fixed-length output**.

The output is usually termed “hash value” or “message digest.”

Widely used in many cryptographic protocols, information security applications such as Digital Signatures and message authentication codes (MACs)

Its main purpose is to check integrity of data send by sender to receiver

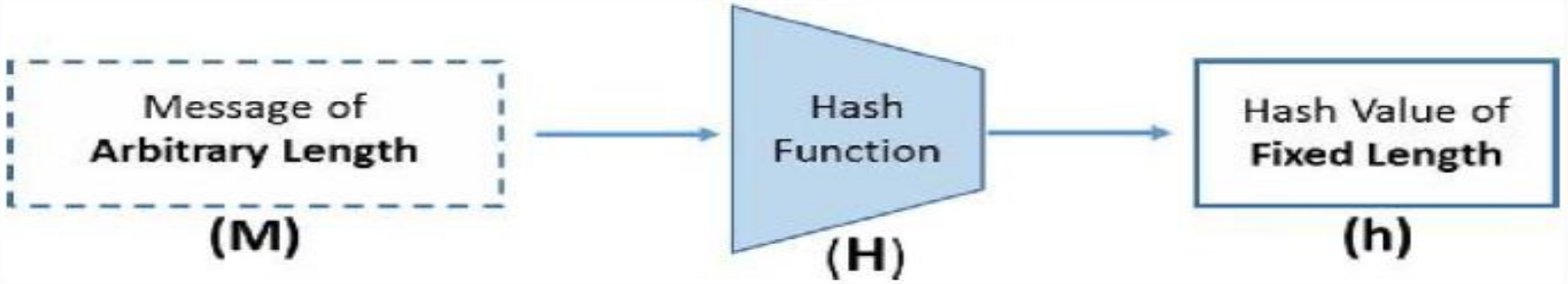
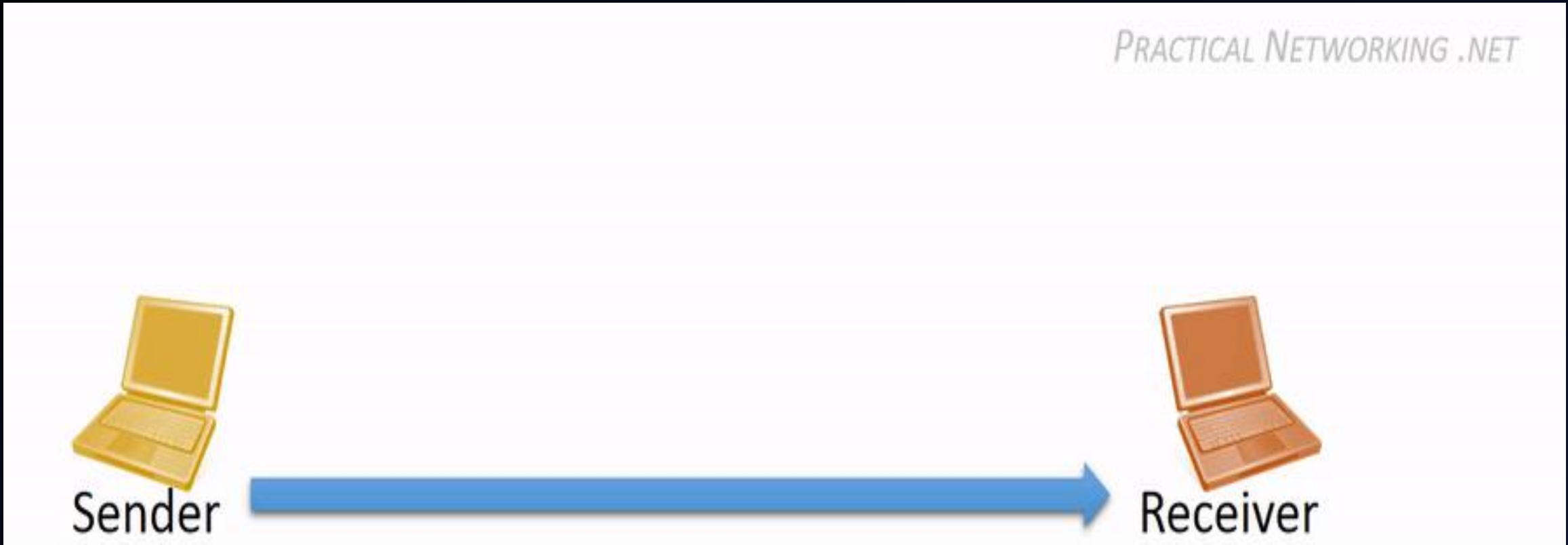
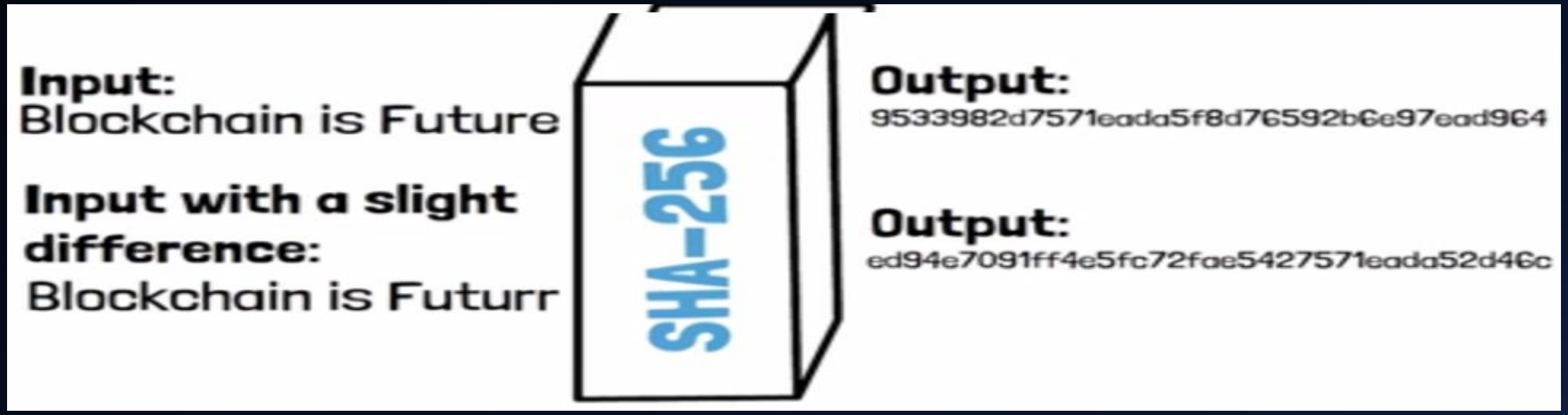



*Figure 2-8* Hash function in its basic form


PRACTICAL NETWORKING .NET
Sender
Receiver

# Core properties of cryptographic hash functions

- Input: Any size; output: Fixed length (e.g., 256/512 bits).
- Hash is fast to compute for any input.
- Deterministic: Same input = same hash every time.
- Hard to reverse: Can’t generate input from hash (except brute force).
- **Avalanche effect**: Small input change = big output change.



# Cryptographic Hash Functions

## Property 2: Hiding

**Definition**: A hash function is hiding if, given output y = H(x), it’s infeasible to determine x.

**Limitation**: If x has only a few possible values (e.g., "heads" or "tails"), an attacker can compute hashes of all possibilities and find x.

**Solution**: x should be chosen from a large, unpredictable set.

**Min-Entropy**: A measure of unpredictability. A high min-entropy means no particular value is likely

**Example**: A 256-bit random value has an extremely low probability (1/2^256) of being guessed

# Cryptographic Hash Functions

## Application of Hiding --- **Commitment**: A Digital Lockbox

**Commitment** -- Way to lock in a value **without revealing it** immediately. Later, you can reveal it to prove what you committed to earlier.

**Real-Life Example: Sealed Envelope Bet**

Imagine you and a friend, Alex, make a bet on a football match. You want to predict the winner **before** the game starts, but you don't want Alex to know your prediction yet.

**Commitment Phase (Sealing the Envelope)**

You **write your prediction** (e.g., "Team A will win") on a piece of paper.

You **seal it in an envelope** and place it on the table where Alex can see it.

Alex knows you made a prediction, but he **doesn't know what it is**.

**Reveal Phase (Opening the Envelope)**

After the game, you **open the envelope** to reveal your prediction.

Alex can see that you **didn't change your prediction** after the game.

# Cryptographic Hash Functions

## Application of Hiding --- Commitment in Cryptography

Instead of using a physical envelope, we use a **cryptographic hash function** to commit to a value digitally.

**Commitment (Locking the Value)**

Choose a **secret message** (e.g., "Team A wins").

Generate a **random number (nonce)** for extra security.

Create a **hash** of (nonce + message) and share the hash publicly.

**Reveal (Unlocking the Value)**

Later, you share the **nonce and the original message**.

Anyone can **recompute the hash** and check that it matches the commitment.

## **Cryptographic Hash Functions** : Property 3: Puzzle-Friendliness

**Puzzle-friendliness** : Means that a hash function makes it very hard to find an input that produces a specific output unless you try many random guesses.

Key Idea: If part of the input is chosen randomly, no shortcut exists to solve the puzzle faster. Only way to find the correct input is by brute-force search (trying many possibilities).

Example: In Bitcoin mining, miners must find a number (nonce) so that hash of the block meets a specific condition. Only way to do this is by trying many random values until one works.

**“friendly" here means "useful for creating hard-to-solve puzzles."**

## **Cryptographic Hash Functions**：Property 3: Puzzle-Friendliness

**What it means**: Hash function is "puzzle-friendly" if, even target output is given , it's extremely hard to find an input that produces that output, especially when part of the input is randomly chosen.

**Example**: Imagine a lottery where you need to guess a number that, when combined with a secret, random number, produces a specific hash. It's nearly impossible to figure out your number without trying almost every possibility.

**Use**: This property is crucial for "search puzzles," where only way to find a solution is to try many different inputs.

**Search Puzzle example**: Puzzle asks  to find an input that makes a hash output start with "0000". If hash is puzzle-friendly, you can't find a shortcut, you have to guess and check.

**Why it matters**: In Bitcoin mining, this principle is used to create a difficult but fair challenge, preventing anyone from gaining an unfair advantage.

## Puzzle-Friendliness in Hash Functions

**Definition**: A hash function H is **puzzle-friendly** if:

For any n-bit output *y*,

When k is chosen randomly (with high min-entropy),

It is extremely hard to find an input x such that $H(k \| x)=y$ in time much less than $2^n$

Intuition : If part of input is chosen randomly, it's extremely difficult to find another input that produces the same output.

**Puzzle-friendliness in real life** : Treasure Hunt Analogy

**Scenario:** Find hidden treasure by solving clues

**Game Master's Rule**:

- Provides a random starting point
- Must follow clues to reach the treasure

**Clues & Hash Functions**:

- Each clue depends on the previous one
- Final clue leads to the treasure

**Puzzle-Friendliness:**

- Random, unpredictable starting point (high min-entropy)
- No shortcut—must solve each clue in order
- Even if you know the treasure’s location, you can't skip clues

Puzzle-Friendliness : Bitcoin Mining

Treasure Hunt: Miners compete to find a "treasure" (a valid block hash).

Random Starting Point: "nonce" (a random number) is like random starting point in treasure hunt.

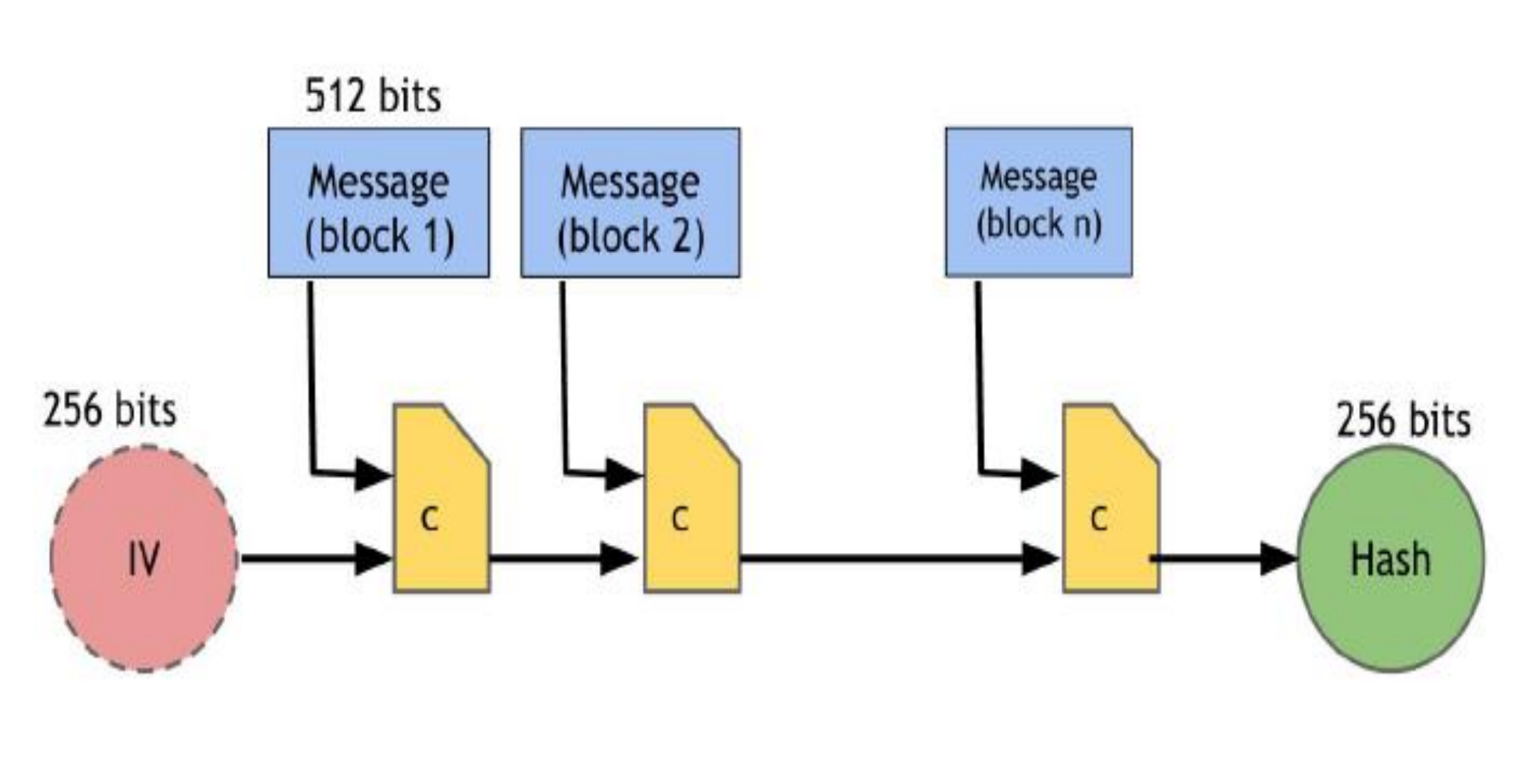
Clues: Miners repeatedly hash the block data with different nonces until they find a hash that meets target (e.g., a hash with a certain number of leading zeros).

No Shortcuts: Only way to find correct nonce is by trying many possibilities (brute force).

**SHA-256 Overview**:

- Widely used hash function, primarily in Bitcoin.
- Works on arbitrary-length inputs using the Merkle-Damgård transform.

- Merkle-Damgård Transform:
  - Converts a fixed-length hash function into one that handles variable-length inputs.
  - Uses a compression function to ensure collision resistance.
- How It Works:
  - Input is divided into m-n length blocks.
  - Each block's output is passed to the next block along with new input.
  - The first block uses an Initialization Vector (IV).
  - The final block's output is the hash result.
- SHA-256 Specifics:
  - Compression function: Takes 768-bit input, produces 256-bit output.
  - Block size: 512 bit




**How SHA-256 processes data to generate a hash**

## Initialization (IV - 256 bits):

- Process starts with a special fixed value called an Initialization Vector (IV).
- This value is predefined and serves as a starting point for hashing.

## Dividing the Message (512-bit Blocks):

- Input message is split into 512-bit chunks (blocks).
- If the message is too short, it is padded to fit the required block size.

## Processing Each Block (Compression Function ‘C’):

- First message block is combined with the IV and processed using a compression function (C).
- Result is passed as input to the next block along with the second message block.
- This process repeats for all blocks.

## Final Hash Output (256 bits):

- Output of last block becomes final 256-bit hash value.
- This hash is unique to the input message and cannot be reversed.

# Hash Pointers and Data Structures

## Hash Pointer: Concept & Function

- A **hash pointer** is a combination of:
  - A **pointer** to the location where data is stored.
  - A **cryptographic hash** of the data at that location.
- Unlike a regular pointer, a hash pointer provides two functions:
  - **Retrieving** the data.
  - **Verifying** that the data hasn't been altered.
- Hash pointers help build **tamper-evident** data structures.

## Hash Pointers in Data Structures

- Any data structure using regular pointers (e.g., linked lists, binary search trees) can be modified to use **hash pointers** instead.
- This provides an added security feature—**data integrity verification.**

# Hash Pointers and Data Structures

## Blockchain: A Tamper-Evident Linked List

- A **blockchain** is a **linked list** where:
  - Each block contains **data**.
  - Instead of a regular pointer to the previous block, it uses a **hash pointer**.
- The **head of the blockchain** is a stored hash pointer that references the most recent block.

## Ensuring Tamper Resistance in Blockchain

- If an attacker modifies a block's data (**block k**), the **hash in block k+1** (which depends on block k) will not match.
- Since **cryptographic hashes are collision-resistant**, any change to the data will result in a completely different hash.
- To hide the tampering, the attacker must also modify:
  - The hash pointer in **block k+1**.
  - The hash pointers in all following blocks up to the **head of the blockchain**.
- However, the **head of the blockchain** is securely stored, making it impossible for the attacker to modify.

# *SHA-256 hash and its construction*

SHA-256 and SHA-512 belongs to family of cryptographic hash functions

Specifically used in Bitcoins!

As the name suggests, it produces a 256-bit hash value, hence the name

Hash functions take arbitrary length input and produce a fixed size output

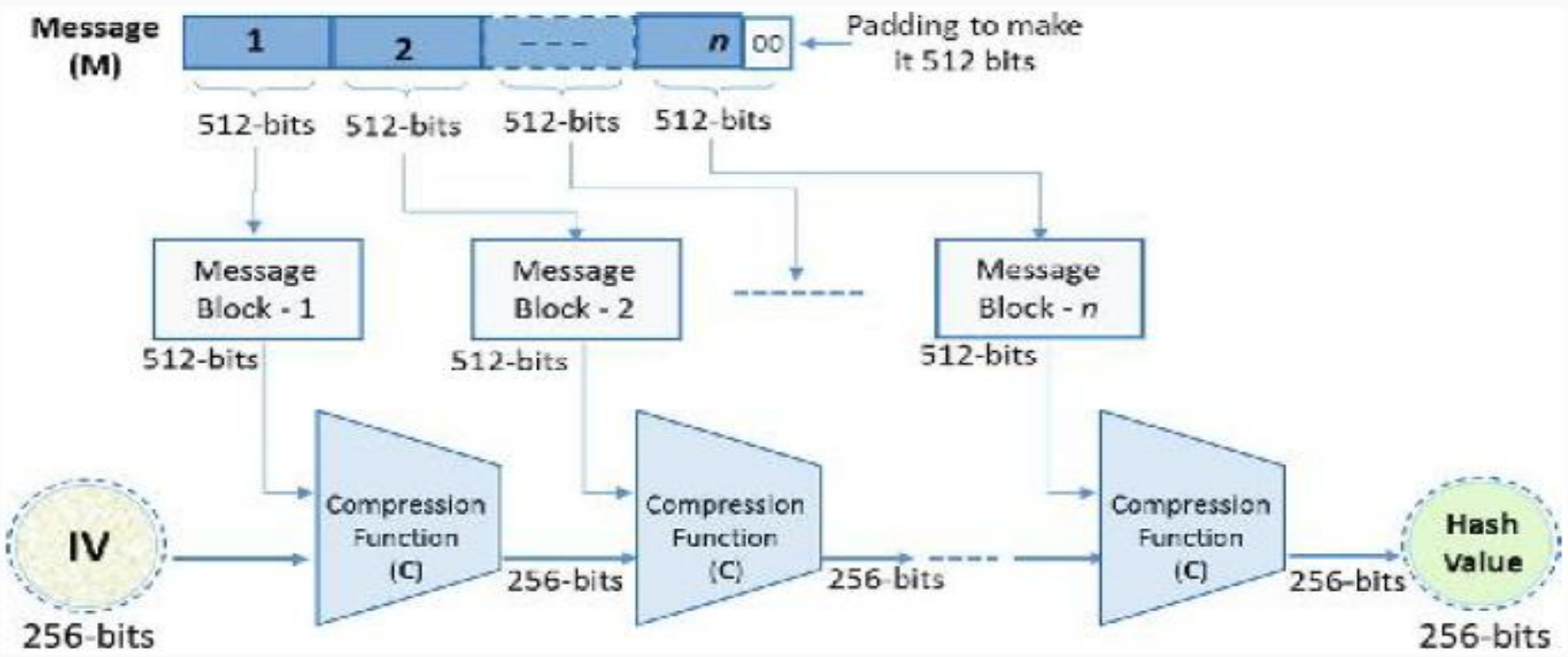



*Figure 2-9* Merkle-Damgård construction for SHA-256

## Merkle Trees

- Merkle tree is another useful data structure ( binary tree) that can be built using hash pointers.
- Data blocks form the leaves of the tree.
- Each pair of blocks is hashed and stored in a parent node.
- This process continues until a single root node remains.

## Security

- The root hash helps verify the integrity of all data.
- If a data block is tampered with, its hash changes, affecting all parent hashes up to the root.
- Since the root hash is stored securely, any modifications can be detected.

## Proof of Membership

- To verify a block is in the tree, only its path to the root needs to be shown.
- Requires only **log(n)** steps for a tree with **n** nodes, making verification efficient.

## Sorted Merkle Trees & Proof of Non-Membership

- A **sorted** Merkle tree orders blocks alphabetically, numerically, or by other criteria.
- Proof of **non-membership** is done by showing the hashes of the blocks **just before and after** the missing block's expected position.

## Hash Pointers in Other Structures

- Hash pointers can be used in any **acyclic** data structure, like **directed acyclic graphs (DAGs)**.
- They enable efficient verification of data integrity.
- Cycles prevent proper hashing, making verification difficult

**Digital Signatures: It** serve as a digital version of handwritten signatures, ensuring:

- **Authenticity**: Only the owner can create the signature, but anyone can verify it.
- **Document Binding**: The signature is tied to a specific document and cannot be transferred to another.

**Digital Signature Scheme : It** consists of three key algorithms:

**1.Key Generation**

- generateKeys(keysize) → (sk, pk) where as (sk) is a secret key and (pk) a public key
- sk is private and used to sign messages and pk is public and used to verify signatures.

**2.Signing a Message**

- sign(sk, message) → sig
- Uses the secret key (sk) to create a signature (sig) for the given message.

**3.Verifying a Signature**

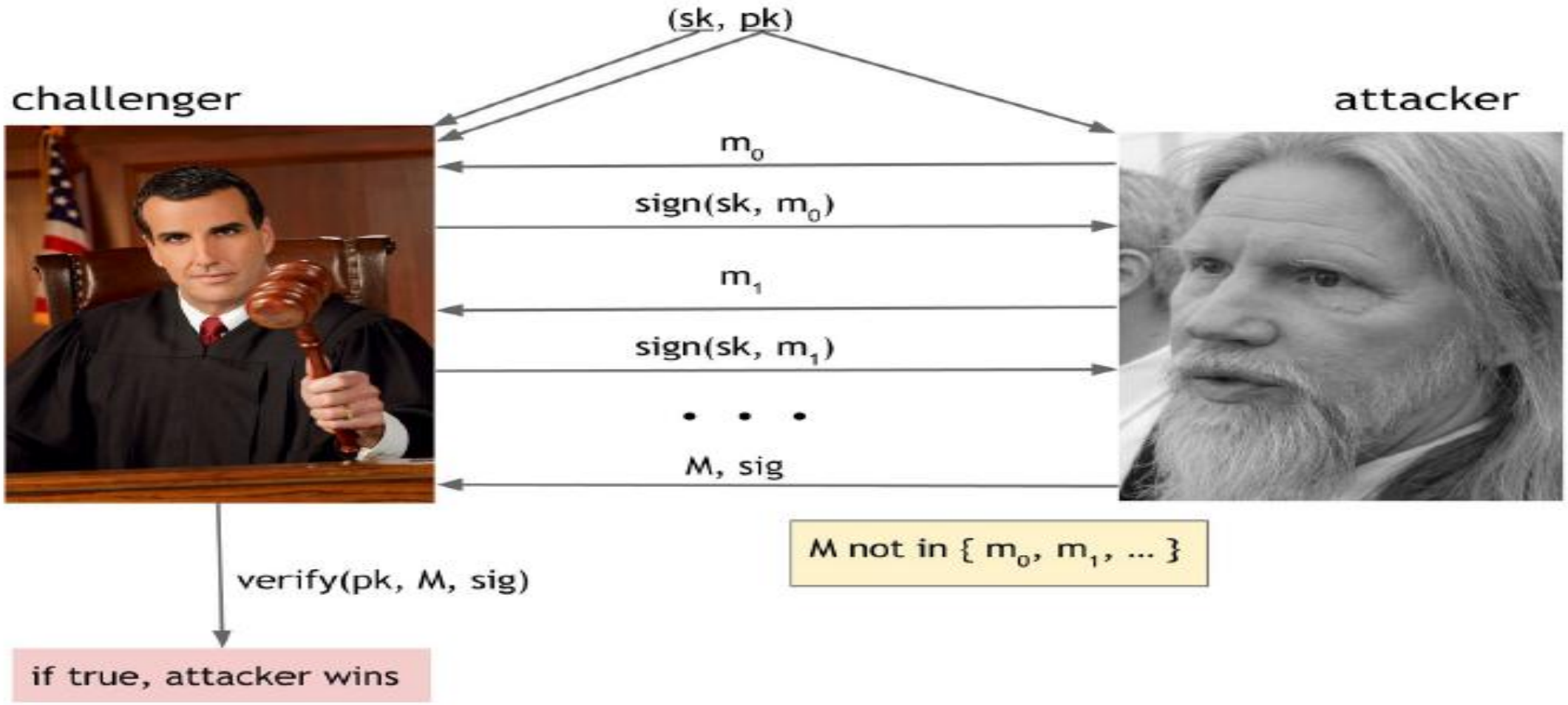
- verify(pk, message, sig) → isValid
- Checks if sig is a valid signature for message using pk and Returns true if valid, false otherwise.

## Required Properties

- **Validity**: A valid signature must verify correctly.
  - verify(pk, message, sign(sk, message)) == true
- **Unforgeability**: It must be computationally infeasible to forge a signature without the secret key.

## Unforgeability Explained

- An attacker may have access to multiple valid signatures.
- The attacker's goal is to forge a new, unseen signature.
- A verification test is performed:
  - If the attacker successfully creates a valid signature for a new message, they win.
  - If they fail, the digital signature scheme remains secure.
- **Key takeaway**: A secure scheme ensures that forging a signature is practically impossible.




**This diagram shows how attacker might try to forge a digital signature without knowing private key.**

## The Challenger (Legitimate User)

- Has a **private key (sk)** and **public key (pk)**.
- Signs multiple messages (**$m_0$, $m_1$, ...**) using the private key, producing signatures (**sign(sk, $m_0$), sign(sk, $m_1$), etc.**).

## The Attacker's Goal

- Attacker sees the signed messages (**$m_0$, $m_1$, ...**) and their signatures.
- They try to **create a new, fake message (M)** that was **never signed before** (so **M ≠ $m_0$, $m_1$, etc.**).
- They also create a **fake signature (sig)** for this message.

## The Attack Succeeds If...

- The attacker tricks the system into accepting their forgery.
- When someone checks the signature using the public key (**verify(pk, M, sig)**), it incorrectly says **"valid"** even though the attacker never legitimately signed **M**

## What This Means

- A secure digital signature scheme should **never** allow this!
- Even if an attacker sees many signed messages, they **should not** be able to forge a signature for a new message.
- If this attack works, the signature scheme is **broken** and unsafe.

**Real-World Example :** Imagine

- You sign checks (**$m_0$, $m_1$**) with a special pen (private key).
- Thief sees your signed checks and tries to **forge a new check (M)** with a fake signature.
- If the bank (verification system) accepts it, the thief wins. A good signature system **must reject fakes**.

**This is why cryptographers design signatures (like ECDSA in Bitcoin) to be unforgeable—seeing past signatures shouldn't help attackers create new ones.**

## Practical Considerations in Digital Signatures

Digital signatures are a crucial part of cryptographic security systems, but their practical implementation requires careful attention to certain key aspects.

**Randomness Matters:**

Digital signature security depends on **strong, unpredictable randomness**; weak randomness can lead to private key compromise.

**Message Size Limitation:**

Since digital signatures work with fixed-length messages, the **solution is to sign the hash of the message instead**.

**Hash Pointers:**

Signing a hash pointer secures **not just the pointer but the entire data structure it references**, ensuring **integrity and tamper resistance** in systems like blockchains.

## ECDSA (Elliptic Curve Digital Signature Algorithm)

- Used in Bitcoin.
- Based on elliptic curves.
- Provides **128-bit security** (equivalent to 2^128 hash function operations).
- Uses the **secp256k1** curve (instead of the more common **secp256r1**).

## Key Sizes in ECDSA

- **Private Key**: 256 bits
- **Public Key (Uncompressed)**: 512 bits
- **Public Key (Compressed)**: 257 bits
- **Message to be Signed**: 256 bits
- **Signature**: 512 bits

## Security Concerns with ECDSA

- **Bad Randomness Can Leak Keys**:
  - Even using bad randomness for a **single signature** can expose the private key.
  - Proper randomness is essential to prevent key leakage.

## Cryptocurrency and Encryption

- **Bitcoin does NOT use encryption**.
- **Why?** Nothing in Bitcoin needs to be hidden or encrypted.
- Instead, Bitcoin relies on **cryptographic techniques like digital signatures and hash functions**.

## 1.4 Public Keys as Identities

### 1. Public Key as an Identity

- A **public key (pk)** can act as an **identity** in a digital signature system.
- If a message is signed and verified under a public key, it means **public key is "saying" the message**.
- Only the person with the corresponding **private key (sk)** can sign messages for that identity.

### 2. Creating New Identities

- We can create a new identity at any time by generating a **new key pair (pk, sk)**.
- Instead of using the full public key, people often use the **hash of pk** as the identity to save space.
- To verify a message:
  1. Check that **pk hashes to the given identity**.
  2. Verify the message signature using **pk**.

## 3. Anonymity of Public Keys

- Public keys **look random** and don’t reveal real-world identities.
- You can create multiple identities and **discard them** when no longer needed.
- Bitcoin follows this system—public key hashes are known as **Bitcoin addresses**.

## 4. Decentralized Identity Management

- Unlike traditional systems, you don’t need a **central authority** to issue an identity.
- You can create multiple identities for **different purposes**.
- Bitcoin and other cryptocurrencies use this method for identity management.

## 5. Security of Unique Identities

•Can someone generate the **same private key** as you?

- Almost **impossible** due to the huge number of possible key combinations (256-bit security).
- However, weak or predictable **randomness** can cause security issues.
- Using a **strong random number generator** is crucial.

## 6. Privacy Limitations

•Even though identities appear random, **behavior patterns** can reveal real identities.

•Transactions linked to an identity over time may allow people to **infer the real person** behind it.

•In Bitcoin, identity isn’t directly tied to your real-world name, but your **actions can reveal who you are**.

# A Simple Cryptocurrency (GoofyCoin)

## Introduction to Cryptocurrencies

- Cryptography is important for making cryptocurrencies secure.
- Hash functions and digital signatures help create and verify digital money.
- Here we will explore a simple cryptocurrency called **GoofyCoin** to explain how it works.

## Rules of GoofyCoin

Creating Coins , Transferring Coins and Spending Coins

**Creating Coins :**

1. Goofy, the creator, can generate new coins anytime.
2. Each coin has a **unique ID**.
3. Goofy creates a statement: **"CreateCoin [uniqueCoinID]"**.
4. He signs this statement with his secret key.
5. The signed statement is the coin.
6. Anyone can verify the coin using Goofy's signature.

## 2.Transferring Coins

1. The owner of a coin can transfer it to someone else.
2. Example: Goofy transfers a coin to Alice.
    1. He creates a statement: **“Pay this to Alice”** (where "this" is the coin reference).
    2. Alice is identified by her **public key**.
    3. Goofy signs the statement.
3. Once signed, the coin now belongs to Alice.

## 3. pending Coins

•Alice can transfer the coin to Bob by:

- Creating a statement: **“Pay this coin to Bob’s public key”**.
- Signing it with her private key.

•Anyone can check the chain of transactions to confirm Bob now owns the coin.

## How Ownership is Verified

- Each coin contains a **chain of signatures** proving ownership.
- A new owner must have a valid signed statement from the previous owner.
- By following the signatures back to the original coin, anyone can verify if the coin is genuine.

# Thank You